

Parallel Computing

Programming with MPI – Part 3

Mariko & Kevin Dunseath

Computational Science and Applications

2024 – 2025

**To learn how to transfer data
between more than
two processes**

Collective communications

Involve ***all*** processes associated with a given communicator, usually `MP I_COMM_WORLD`

Different types:

synchronisation: forcing processes to wait until all members of the communicator group have reached a given point in the execution

data sharing: distributing data from one process to all the others, or gathering data from all processes onto one process

computation: one process collects data from all the others, and performs a simple operation (addition, multiplication) on this data. Also known as **reduction**

Collective communications are blocking

Barrier

Creates a barrier, at which each task waits until all processes arrive at the same call

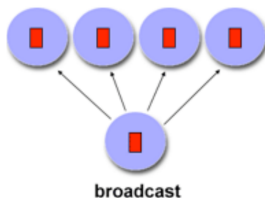
```
MPI_Barrier(comm)
```

Data sharing (1)

Broadcast

A message is sent by the process with rank "root" and received by all other processes in the same communicator

```
MPI_Bcast (buffer, len, type, root, comm)
```



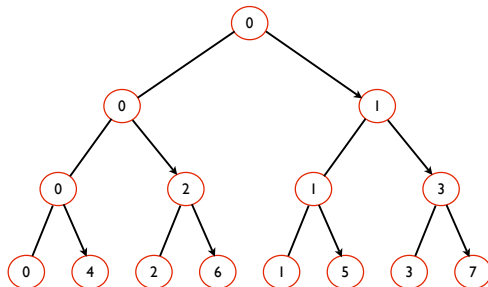
https://computing.llnl.gov/tutorials/mpi/images/collective_comm.gif

Important: Collective communication routines are executed by all processes, not only by root!

```
int nprocs,rank;
float x;
MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&nprocs);
if (rank == 0) {
    printf("Enter a real number");
    scanf("%f",&x);
    printf("\n");
}
MPI_Bcast(&x,1,MPI_FLOAT,0,MPI_COMM_WORLD);
printf("Process %d has x = %f \n",rank,x);
MPI_Finalize();
```

Broadcast

Example: Broadcast using a tree structure



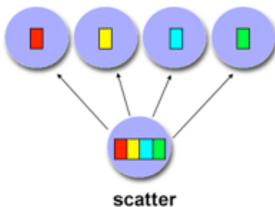
- `MPI_Bcast` may be optimized for topology of machine
- more efficient than several calls to `MPI_Send` and `MPI_recv`

Data sharing (3)

Scatter

Distributes different messages (data) of identical lengths from a single source to all other processes in the same communicator

```
MPI_Scatter (&sendbuf, sendlen, sendtype, &recvbuf,  
            recvlen, recvtype, root, comm)
```



https://computing.llnl.gov/tutorials/mpi/images/collective_comm.gif

MPI_Scatter

```
int nprocs, rank, root;
int rbuff[100];
int* sbuff;

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

if (rank==root) {
    sbuff = (int *)malloc(nprocs*100*sizeof(int));
    ...
}

MPI_Scatter(sbuff, 100, MPI_INT, rbuff, 100,
            MPI_INT, root, MPI_COMM_WORLD);}
```

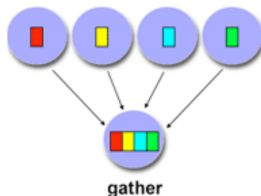
- Data stored in `sendbuf` are split into segments, each of which has `sendlen` elements of type `sendtype`. The first segment is sent to process 0, the second to process 1, ...
- `recvlen` and `recvtype` specify the number and type of elements received by each process. They will usually be the same as `sendlen` and `sendtype`

Data sharing (4)

Gather

Gathers different messages (data) of identical lengths from each process in the communicator onto a single process "root"

```
MPI_Gather (&sendbuf, sendlen, sendtype, &recvbuf,  
           recvlen, recvtype, root, comm)
```



https://computing.llnl.gov/tutorials/mpi/images/collective_comm.gif

MPI_Gather

```
int nprocs, rank, root;
int sbuff[100];
int* rbuff;

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

if (rank==root) {
    rbuff = (int *)malloc(nprocs*100*sizeof(int));
    ...
}
MPI_Gather(sbuff, 100, MPI_INT, rbuff, 100,
           MPI_INT, root, MPI_COMM_WORLD);
```

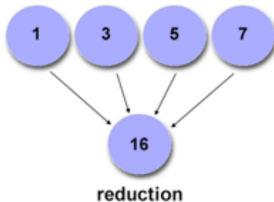
- Data referenced by `sendbuf` on each process consists of `sendlen` elements of type `sendtype`
- `recvlen` and `recvtype` specify the number and type of elements **received from each process, not the total amount of data received**. They will usually be the same as `sendlen` and `sendtype`
- Data is stored in `recvbuf` on the process `root` in process order: data from process 0 followed by data from process 1 followed by data from process 2 ...

Collective communications routines

Reduce

Applies a reduction to the data on all processes, storing the result on the process with rank "root"

```
MPI_Reduce (&sendbuf, &recvbuf, len, type, op, root,  
            comm)
```



https://computing.llnl.gov/tutorials/mpi/images/collective_comm.gif

Reduction operations op include

`MPI_MAX`, `MPI_MIN`, `MPI_SUM`, `MPI_PROD`, `MPI_MAXLOC`,
`MPI_MINLOC`, `MPI_LAND`, `MPI_LOR`

as well as a number of bit-wise operations

Collective communications routines

AllGather

As for `MPI_Gather`, except that the result is available to all processes

```
MPI_Allgather(&sendbuf, sendlen, sendtype,  
              &recvbuf, recvlens, recvtype, comm)
```

AllReduce

As for `MPI_Reduce`, except that the result is available to all processes.

```
MPI_Allreduce(&sendbuf, &recvbuf, len, type, op,  
              comm)
```

Equivalent to `MPI_Gather/_Reduce` followed by `MPI_Bcast`

MPI_Sendrecv

`MPI_Sendrecv` combines separate send and receive operations in one function call

A message can be sent to one process (destination) and a different message received from another process (source)
The source and destination processes may be the same

The sent and received messages must be stored in different, non-overlapping arrays

```
int err = MPI_Sendrecv(sbuffer, slen, stype, dest, stag,  
                        rbuffer, rlen, rtype, src, rtag,  
                        comm, &status)
```

The send and receive operations are blocking, and can help avoid deadlocks

Message probing

In `MPI_Recv`, the message length must be known beforehand, in order to allocate enough space in the receive buffer

The length/dimensions of an array can be sent in an earlier message, via `MPI_Send`:

```
if (rank==0) {  
    // Set up message in data array  
    MPI_Send(&len,1,MPI_INT,1,0,MPI_COMM_WORLD);  
    MPI_Send(data,len,MPI_DOUBLE,1,0,MPI_COMM_WORLD);  
} else if (rank==1) {  
    int len;  
    MPI_Status status;  
    MPI_Recv(&len,1,MPI_INT,0,0,MPI_COMM_WORLD,&status);  
    double* data = new double [len];  
    MPI_Recv(data,len,MPI_DOUBLE,0,0,MPI_COMM_WORLD,&status);  
}
```

Message probing (2)

`MPI_Probe` (blocking) or `MPI_Iprobe` (non-blocking) allow the user to check an incoming message without actually receiving it

```
int err = MPI_Probe(src, tag, comm, &status)
int err = MPI_Iprobe(src, tag, comm, &flag, &status)

int err = MPI_Get_count(&status, type, &count)
```

`MPI_Get_count` returns the number of values of the specified data type that are to be received

Message probing (3)

Example determining the length of the message in order to allocate the resources necessary to receive it

```
if (rank==0) {  
    // Set up message in data array  
    MPI_Send(data, len, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD);  
} else if (rank==1) {  
    int len;  
    MPI_Status status;  
    MPI_Probe(0, 0, MPI_COMM_WORLD, &status);  
    MPI_Get_count(&status, MPI_DOUBLE, &len);  
    double* data = new double [len];  
    MPI_Recv(data, len, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, &status);  
}
```

`MPI_Probe` blocks until a matching message (source, tag) has been found

MPI predefined constants

MPI also defines a large number of constants (in `mpi.h`)

For example:

`MPI_COMM_WORLD`: default communicator containing all processes

`MPI_MAX_PROCESSOR_NAME`: minimum size for buffer containing the result of `MPI_Get_processor_name`

`MPI_SUCCESS`: result returned by MPI routines on successful completion

`MPI_ANY_SOURCE`: wildcard indicating that the message to be received can come from any source

`MPI_ANY_TAG` wildcard indicating that the label or tag on the message to be received can have any value

Tutorial: Loop

Write a program which sends `nloop` times a message of `N` doubles in a loop of `nprocs` processes:

- 1 Process 0 asks the user to enter the values of `nloop` and `N` which are broadcast to the other processes.
- 2 Process 0 initializes an array of `N` doubles with random values between 0 and 1.
- 3 Process 0 sends the array with `tag=0` to process 1; after receiving the message, process 1 sends it to process 2 and so on ; process `nprocs-1` sends it to process 0. Repeat the loop `nloop` times.
- 4 Process 0 prints the average time per loop taken by the message to go round.
- 5 Modify the code so that `nloop` is no longer broadcast: for the last loop, process 0 changes the `tag` to 1, the program ends when process 0 receives a message with tag 1.
- 6 Run the code for different values of `nloop` and `N` and represent the average time/loop in a graph.

Tutorial: Trace of a distributed matrix

Write a code where

- 1 Process 0 asks for the dimension `ndim` of a square matrix.
- 2 The integer `ndim` is broadcast.
- 3 The square matrix is split among the processes in blocks of `nrow` rows. If `ndim` is not a multiple of the number `nprocs` of processes, the first `ndim%nprocs` processes will have one row more than the others.
Example: if `ndim=100` and `nprocs=8`, the first four processes will store 13 rows while the last 4 processes will store 12.
- 4 Each process initializes its local matrix with random elements between -1 and 1.
- 5 Each process calculates the contribution of its elements to the trace of the global matrix.
- 6 Process 0 collects all contributions to evaluate and print the trace of the global matrix.